Lec5:
Distributed Database Design

ا.سعاد محمد عثمان

Designing distributed databases

Two major strategies that have been identified for designing distributed databases are :

- Top-down approach
- Bottom-up approach.

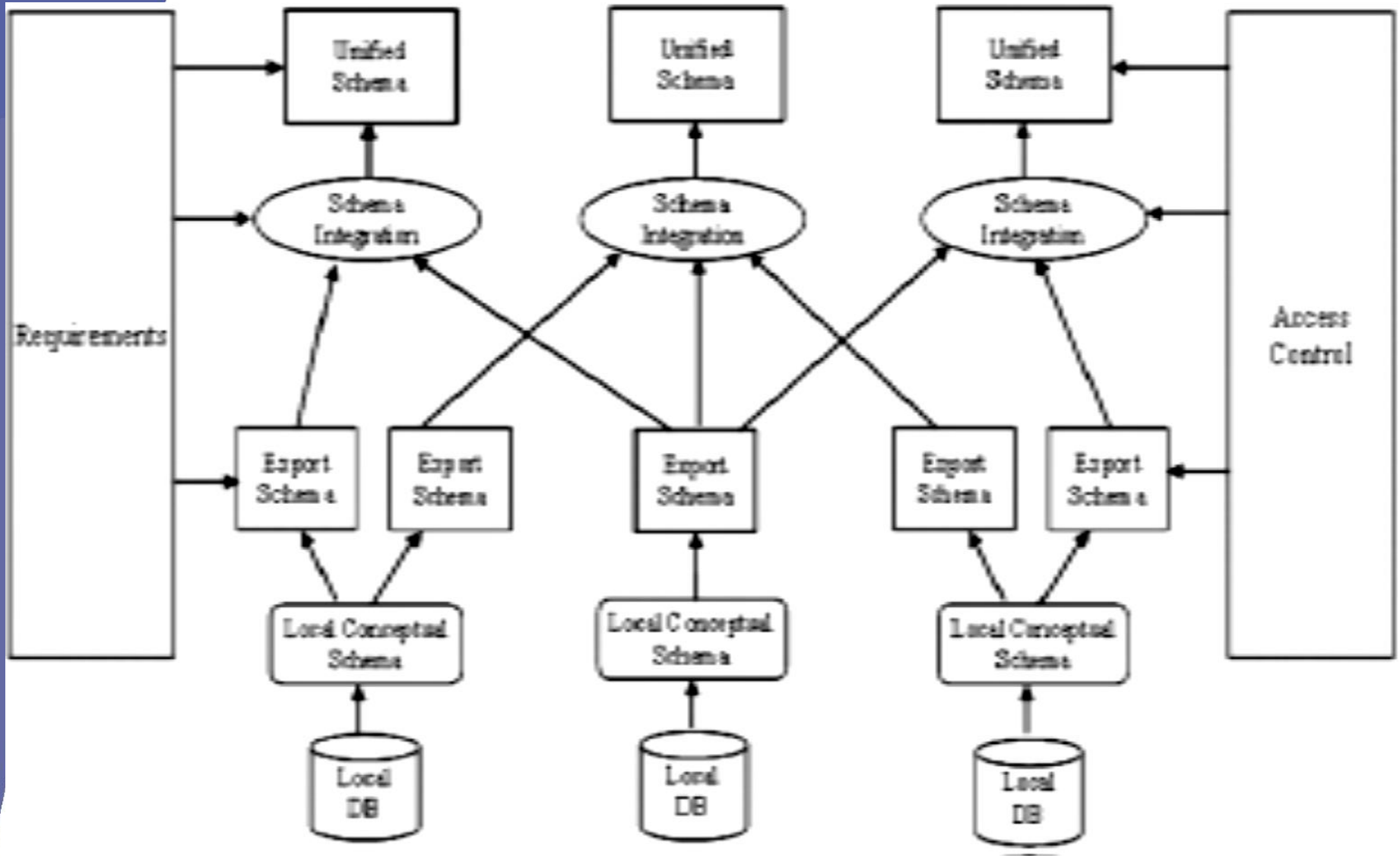
Bottom-up approach

- A **bottom–up** design approach considers the existing data distributed within an organization and uses a process called **schema integration** to create at least one unified schema.
- **Schema integration** is a process that uses a collection of existing conceptual model elements, which have previously been exported from one or more LCMs, to generate a semantically integrated model (a single, unified schema).

Bottom-up approach(cont.)

- **Unified Schema** An integrated, nonredundant, and consistent schema for a set of local database systems.

Bottom-up approach



Distribution Design activity

- The goal of any data distribution is to provide for increased availability, reliability, and improved query access time.
- The distribution design activity are:
  - Allocation (Non Replicated, Non Fragmented)
  - Fragmentation
  - Replication.
  - Mixed

Allocation

- **Allocation** allows a designer to place different tables of a given database at different sites.
- The idea is that data should be placed close to (or at the site) where it is needed the most.
- The benefit of this approach is the reduction of the communication component of the processing cost.

Fragmentation

- Fragmentation design approach breaks a table up into two or more pieces called fragments or partitions and allows storage of these pieces in different sites.
- Fragmentation provides for increased parallelism, access, disaster recovery, and security/privacy

Fragmentation

- There are three alternatives to fragmentation:
  - Vertical fragmentation
  - Horizontal fragmentation
  - Hybrid fragmentation.
- Horizontal fragmentation can further be classified into two classes:
  - Primary horizontal fragmentation (PHF)
  - Derived horizontal fragmentation (DHF).

Fragmentation

- When thinking about fragmentation, designers need to decide on the **degree of granularity** for each fragment. In other words, how many of the table columns and/or rows should be in a fragment

Vertical Fragmentation

- Vertical fragmentation (VF) will group the columns of a table into fragments.
- VF must be done in such a way that the original table can be reconstructed from the fragments.
- This fragmentation requirement is called “reconstructiveness.” This requirement is used to reconstruct the original table when needed.

EXAMPLE

EmpID	Name	Loc	Sal	DOB	Dept
283948	Joe	LA	25,000	2/6/43	Maintenance
109288	Larry	New York	35,200	12/3/52	Payroll
284003	Moe	LA	43,000	7/12/56	Maintenance
320021	Sam	New York	53,500	8/30/47	Production
123456	Steve	Minneapolis	67,000	5/14/78	Management
334456	Jack	New York	55,000	5/30/67	Production
222222	Saeed	Minneapolis	34,000	4/27/59	Management

EMP Table

VF EXAMPLE

- **Example:**

```
Create table EMP_SAL as
Select EmpID, Sal
From EMP;
```

```
Create table EMP_NON_SAL as
Select EmpID, Name, Loc, DOB, Dept
 From EMP;
```

Fragmentation transparency

- After fragmentation, the EMP table will not be stored physically anywhere. But, to provide for fragmentation transparency—not requiring the users to know that the EMP table is fragmented. To do this, we will use the join statement anywhere the EMP table is required.

EXAMPLE

```
Select EMP_SAL.EmpID, Sal, Name, Loc, DOB, Dept
From EMP_SAL, EMP_NON_SAL
Where EMP_SAL.EmpID = EMP_NON_SAL.EmpID;
```

Horizontal Fragment

- Horizontal Fragment (HF) will group the rows of a table based on the values of one or more columns.
- Every horizontal fragment must have all columns of the original base table.
- To create a horizontal fragment from a table, a select statement is used. For example, the following statement selects the row from R satisfying condition C:

Select * from R where C;

Primary Horizontal Fragmentation

- **Primary Horizontal Fragmentation** (PHF) partitions a table horizontally based on the values of one or more columns of the table.
- **Example :** Consider the EMP table. Suppose we have three branch offices, with each employee working at only one office. For ease of use, we decide that information for a given employee should be stored in the DBMS server at the branch office where that employee works.

Example

```
Create table MPLS_EMPS as Select *
From EMP
Where Loc = ‘Minneapolis’;
```

```
Create table LA_EMPS as Select *
From EMP
Where Loc = ‘LA’;
```

```
Create table NY_EMPS as Select *
From EMP
Where Loc = ‘New York’;
```

Horizontal Fragmentation Transparency

- After fragmentation, the EMP table will not be physically stored anywhere.
- To provide for horizontal fragmentation transparency, we have to be able to reconstruct.
- The EMP table from its HF fragments. To do this, we will use the union statement anywhere the EMP table is required.

Example

```
Select * from MPLS_EMPS
Union
Select * from LA_EMPS
Union
Select * from NY_EMPS;
```

Derived Horizontal Fragmentation

- **Derived Horizontal Fragmentation (DHF)** is fragment a table according to the way that another table is fragmented. DHF A horizontal fragmentation approach that fragments rows of a table based on the values of columns of another table.
- DHF is usually used for two tables that are naturally (and frequently) joined. Therefore, storing corresponding fragments from the two tables at the same site will speed up the join across the two tables.

Example

```
Create table PROJ1 as
Select Pno, Pname, Budget, PROJ.Dno
From PROJ, MPLS_DEPTS
Where PROJ.Dno = MPLS_DEPTS.Dno;
```

```
Create table PROJ2 as
Select Pno, Pname, Budget, PROJ.Dno
From PROJ, NY_DEPTS
Where PROJ.Dno = NY_DEPTS.Dno;
```

```
Create table PROJ3 as
Select Pno, Pname, Budget, PROJ.Dno
From PROJ, LA_DEPTS
Where PROJ.Dno = LA_DEPTS.Dno;
```

Hybrid Fragmentation

- **Hybrid fragmentation** (HyF) uses a combination of horizontal and vertical fragmentation to generate the fragments we need.
- There are two approaches to doing this. In the first approach, we generate a set of horizontal fragments and then vertically fragment one of more of these horizontal fragments. In the second approach, we generate a set of vertical fragments and then horizontally fragment one or more of these vertical fragments.

Hybrid Fragmentation

- This fragmentation approach provides for the most flexibility for the designers but at the same time it is the most expensive approach with respect to reconstruction of the original table.

Example

```
Create table NON_SAL_MPLS_EMPS as
Select EmpID, Name, Loc, DOB, Dept
From EMP
Where Loc = ‘Minneapolis’;
```

```
Create table NON_SAL_LA_EMPS as
Select EmpID, Name, Loc, DOB, Dept
From EMP
Where Loc = ‘LA’;
```

```
Create table NON_SAL_NY_EMPS as
Select EmpID, Name, Loc, DOB, Dept
From EMP
Where Loc = ‘New York’;
```

Replicated

- **Replicated** is the more copies of a table/fragment one creates, the easier it is to query that table/fragment. On the other hand, the more copies that exist, the more complicated (and time consuming) it is to update all the copies.
- **There are tow alternative of replicated**:
  - Fully replicated.
  - Partially replicated.

Replicated

- **Fully Replicated** This design alternative stores one copy of each database table at every site. provides for the best possible query performance.
- **Partially Replicated** In this distribution alternative, the designer will make copies of some of the tables (or fragments) in the database and store these copies at different sites. This is based on the belief that the frequency of accessing database tables is not uniform.